

OSSP-SKILL-WEEK11

SESSION-21

Task1: To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components

Source code:

```
#include <stdio.h>
#include <string.h>

int parse_command(char *input, char *command, char *argument)
{
    char *space;

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
        return 0;

    space = strchr(input, ' ');

    if (space != NULL)
    {
        *space = '\0';
        strcpy(command, input);
        strcpy(argument, space + 1);
    }
    else
    {
        strcpy(command, input);
        argument[0] = '\0';
    }

    return 1;
}
```

```
GNU nano 7.2
#ifndef SESSION21_PARSER_H
#define SESSION21_PARSER_H

int parse_command(char *input, char *command, char *argument);

#endif
```

GNU nano 7.2

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#include "session21_parser.h"

int main()
{
    char input[100];
    char command[50];
    char argument[50];

    printf("Skill Week 11 - Session 21 - Task 1\n");
    printf("-----\n");
    printf("Enter commands such as:\n");
    printf("  ls\n");
    printf("  pwd\n");
    printf("  echo Hello\n");
    printf("  exit\n\n");

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        if (!parse_command(input, command, argument))
        {
            printf("Error: Empty command.\n");
            continue;
        }

        if (strcmp(command, "exit") == 0)
        {
            printf("Shell exiting...\n");
            break;
        }
    }
}
```

```

        break;
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        continue;
    }

    if (pid == 0)
    {
        if (strlen(argument) > 0)
        {
            execlp(command, command, argument, NULL);
        }
        else
        {
            execlp(command, command, NULL);
        }

        perror("Execution failed");
        exit(1);
    }

    waitpid(pid, NULL, 0);

    printf("Command execution completed.\n");
}

return 0;
}

```

Output:

```

root@LAPTOP-1COJKKG6:~# nano session21_parser.c
root@LAPTOP-1COJKKG6:~# nano session21_parser.h
root@LAPTOP-1COJKKG6:~# nano session21_task1.c
root@LAPTOP-1COJKKG6:~# gcc session21_task1.c session21_parser.c -o session21_task1
root@LAPTOP-1COJKKG6:~# ./session21_task1
Skill Week 11 - Session 21 - Task 1
-----
Enter commands such as:
    ls
    pwd
    echo Hello
    exit

myshell> pwd
/root
Command execution completed.
myshell> echo Hello
Hello
Command execution completed.
myshell> exit
Shell exiting...
root@LAPTOP-1COJKKG6:~# |

```

Observation: I learned how different modules can be connected through header files and function interfaces. I understood how parsing, process creation, command execution, and synchronization work together in an end-to-end shell.

Task2: Develop a shell program that detects runtime errors, generates meaningful error messages, logs failures, handles invalid commands/syntax, and recovers gracefully from error conditions.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>
#include <time.h>

void log_error(const char *message)
{
    FILE *file = fopen("session21_errors.log", "a");

    if (file == NULL)
    {
        perror("Log file error");
        return;
    }

    time_t now = time(NULL);

    fprintf(file, "[%s] %s\n",
            ctime(&now),
            message);

    fclose(file);
}

int main()
{
    char command[100];

    printf("Skill Week 11 - Session 21 - Task 2\n");
    printf("-----\n");
    printf("Test valid and invalid commands.\n");
    printf("Use 'exit' to quit.\n\n");

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';
    }
}
```

```

if (strlen(command) == 0)
{
    printf("Error: Empty command.\n");
    log_error("Empty command entered.");
    continue;
}

if (strcmp(command, "exit") == 0)
{
    printf("Shell exiting...\n");
    break;
}

pid_t pid = fork();

if (pid < 0)
{
    perror("fork failed");
    log_error("fork() failed.");
    continue;
}

if (pid == 0)
{
    execl("/bin/sh", "sh", "-c", command, NULL);

    perror("Command execution failed");
    exit(127);
}

int status;

if (waitpid(pid, &status, 0) == -1)
{
    perror("waitpid failed");
    log_error("waitpid() failed.");
    continue;
}

if (WIFEXITED(status))
{
    int exit_status = WEXITSTATUS(status);

    if (exit_status != 0)
    {

```

```

    if (waitpid(pid, &status, 0) == -1)
    {
        perror("waitpid failed");
        log_error("waitpid() failed.");
        continue;
    }

    if (WIFEXITED(status))
    {
        int exit_status = WEXITSTATUS(status);

        if (exit_status != 0)
        {
            printf("Error: Command failed with status %d.\n",
                exit_status);

            log_error("Command execution returned an error.");
        }
        else
        {
            printf("Command executed successfully.\n");
        }
    }
    else if (WIFSIGNALED(status))
    {
        printf("Error: Process terminated by signal %d.\n",
            WTERMSIG(status));

        log_error("Process terminated by a signal.");
    }
}

return 0;
}

```

Output:

```

root@LAPTOP-1COJKKG6:~# nano session21_task2.c
root@LAPTOP-1COJKKG6:~# gcc session21_task2.c -o session21_task2
root@LAPTOP-1COJKKG6:~# ./session21_task2
Skill Week 11 - Session 21 - Task 2
-----
Test valid and invalid commands.
Use 'exit' to quit.

myshell> pwd
/root
Command executed successfully.
myshell> abcxyz
sh: 1: abcxyz: not found
Error: Command failed with status 127.
myshell>
Error: Empty command.
myshell> exit
Shell exiting...
root@LAPTOP-1COJKKG6:~# |

```

Observation: I learned how to detect runtime errors and invalid commands, generate meaningful error messages, and log failures. I also learned how a shell can recover from errors and continue execution without terminating.

Session -22

Task1: Develop a C program with dynamic memory allocation, analyze it using Valgrind, identify memory leaks, fix the memory errors, release resources, and verify proper cleanup.

Source code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    printf("Skill Week 11 - Session 22 - Task 1\n");
    printf("-----\n");

    char *name;
    int *numbers;

    name = malloc(50 * sizeof(char));
    numbers = malloc(5 * sizeof(int));

    if (name == NULL || numbers == NULL)
    {
        printf("Memory allocation failed.\n");

        free(name);
        free(numbers);

        return 1;
    }

    strcpy(name, "OSSP Memory Test");

    for (int i = 0; i < 5; i++)
    {
        numbers[i] = (i + 1) * 10;
    }

    printf("Name: %s\n", name);
    printf("Numbers: ");

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", numbers[i]);
    }
```

```

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", numbers[i]);
    }

    printf("\n");

    /*
     * Release allocated memory.
     */
    free(name);
    free(numbers);

    printf("All allocated memory released.\n");

    return 0;
}
```

Output:

```

root@LAPTOP-1COJKKG6:~# nano session22_task1.c
root@LAPTOP-1COJKKG6:~# gcc -g session22_task1.c -o session22_task1
root@LAPTOP-1COJKKG6:~# ./session22_task1
Skill Week 11 - Session 22 - Task 1
-----
Name: OSSP Memory Test
Numbers: 10 20 30 40 50
All allocated memory released.
root@LAPTOP-1COJKKG6:~# valgrind --leak-check=full ./session22_task1
==1476== Memcheck, a memory error detector
==1476== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1476== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1476== Command: ./session22_task1
==1476==
Skill Week 11 - Session 22 - Task 1
-----
Name: OSSP Memory Test
Numbers: 10 20 30 40 50
All allocated memory released.
==1476==
==1476== HEAP SUMMARY:
==1476==   in use at exit: 0 bytes in 0 blocks
==1476== total heap usage: 3 allocs, 3 frees, 1,094 bytes allocated
==1476==
==1476== All heap blocks were freed -- no leaks are possible
==1476==
==1476== For lists of detected and suppressed errors, rerun with: -s
==1476== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
root@LAPTOP-1COJKKG6:~# |

```

Observation: I learned how to use Valgrind to detect memory leaks and memory errors in a C program. I understood the importance of releasing dynamically allocated memory using `free()` and verifying proper memory cleanup using Valgrind.

Task2: Design test cases, create a test script, automate program execution, verify outputs, compare expected and actual results, and generate a test report.

Source code:

```
GNU nano 7.2
#!/bin/bash

echo "Skill Week 11 - Session 22 - Task 2"
echo "-----"

echo "Test 1: pwd"

echo "pwd" | ./session21_task2 > output1.txt

if grep -q "Command executed successfully" output1.txt
then
    echo "Test 1: PASS"
else
    echo "Test 1: FAIL"
fi

echo
echo "Test 2: echo command"

echo -e "echo Hello\nexit" | ./session21_task2 > output2.txt

if grep -q "Hello" output2.txt
then
    echo "Test 2: PASS"
else
    echo "Test 2: FAIL"
fi

echo
echo "Test 3: Invalid command"

echo -e "invalid_command_xyz\nexit" | ./session21_task2 > output3.txt

if grep -q "Error" output3.txt
then
    echo "Test 3: PASS"
else
    echo "Test 3: FAIL"
fi
```

```

        echo "Test 3: FAIL"
    fi

    echo
    echo "Test 4: Exit command"

    echo "exit" | ./session21_task2 > output4.txt

    if grep -q "Shell exiting" output4.txt
    then
        echo "Test 4: PASS"
    else
        echo "Test 4: FAIL"
    fi

    echo
    echo "Generating test report..."

    {
        echo "Session 22 - Automated Test Report"
        echo "=====
        echo "Test 1 - pwd                : PASS/FAIL checked above"
        echo "Test 2 - echo                : PASS/FAIL checked above"
        echo "Test 3 - Invalid command : PASS/FAIL checked above"
        echo "Test 4 - exit                  : PASS/FAIL checked above"
    } > test_report.txt

    echo "Test report generated: test_report.txt"

```

Output:

```

root@LAPTOP-1COJKKG6:~# nano session22_task2.sh
root@LAPTOP-1COJKKG6:~# chmod +x session22_task2.sh
root@LAPTOP-1COJKKG6:~# ls session21_task2
session21_task2
root@LAPTOP-1COJKKG6:~# ./session22_task2.sh
Skill Week 11 - Session 22 - Task 2
-----
Test 1: pwd
Test 1: PASS

Test 2: echo command
Test 2: PASS

Test 3: Invalid command
sh: 1: invalid_command_xyz: not found
Test 3: PASS

Test 4: Exit command
Test 4: PASS

Generating test report...
Test report generated: test_report.txt
root@LAPTOP-1COJKKG6:~# cat test_report.txt
Session 22 - Automated Test Report
=====
Test 1 - pwd                : PASS/FAIL checked above
Test 2 - echo                : PASS/FAIL checked above
Test 3 - Invalid command : PASS/FAIL checked above
Test 4 - exit                  : PASS/FAIL checked above
root@LAPTOP-1COJKKG6:~# |

```

Observation: I learned how to design test cases and automate program testing using a shell script. I understood how to compare expected and actual outputs, identify PASS or FAIL results, and generate a test report.